

Tokenization: The Fuel of Language Models

What Is Tokenization?

Computers cannot process raw text — they only understand numbers. Tokenization is the process of breaking text into small units (tokens) and converting each token to a numerical ID.

Example: 'unforgettable' → ['un', 'forget', 'table'] → [1205, 8507, 2420]

Why Not Just Split by Words?

Splitting by words creates problems:

- Vocabulary explosion: millions of unique words, including misspellings and rare words.
- No sub-word structure: 'running', 'runner', 'runs' share a root but look unrelated.
- Out-of-vocabulary (OOV) problem: new words break the system.

Sub-word tokenization (Byte-Pair Encoding — BPE) solves these issues by splitting rare words into known sub-word pieces.

The Tokenization Pipeline

- 1. Text input: 'The cat sat on the mat.'
- 2. Tokenize: ['The', ' cat', ' sat', ' on', ' the', ' mat', '.']
- 3. Convert to IDs: [464, 4758, 9178, 319, 262, 2603, 13]
- 4. Embed: each ID → a high-dimensional vector (e.g., 768 dimensions).
- 5. Feed vectors into the model.

Note: spaces are often part of the token (' cat' not 'cat'). GPT-4 uses ~100,000 token vocabulary.

Why Tokenization Matters for Prompt Engineering

- Pricing is per token, not per character — efficiency saves cost.
- Context window limits are in tokens, not words (approx. 0.75 words per token).
- Unusual punctuation, code, and non-English text tokenize less efficiently (more tokens per word).

Rule of thumb: 1 token $\approx$ 4 characters $\approx$ 0.75 English words.

From Tokens to Embeddings

After tokenization, each token ID is mapped to an embedding vector — a list of hundreds of floating-point numbers that encodes the token's meaning. Similar tokens have similar vectors.

- 'king' - 'man' + 'woman' $\approx$ 'queen' (famous word2vec result).
- Embeddings capture semantic relationships in high-dimensional space.
- The model learns embeddings during pre-training.

Key Takeaways

- Tokenization converts text $\rightarrow$ sub-word tokens $\rightarrow$ numerical IDs $\rightarrow$ embedding vectors.
- Sub-word BPE tokenization handles rare words and new vocabulary gracefully.
- Context window and API costs are measured in tokens, not words.
- Embeddings encode semantic meaning in numerical vector space.

Quick-Revision Questions

- Tokenize the word 'unhappiness' into likely sub-word pieces.
- Why is sub-word tokenization preferred over word-level tokenization?
- What is an embedding vector and what does it represent?